

fspecial

Create predefined 2-D filter

Syntax

```
h = fspecial(type)
h = fspecial('average',hsize)
h = fspecial('disk',radius)
h = fspecial('gaussian',hsize,sigma)
h = fspecial('laplacian',alpha)
h = fspecial('log',hsize,sigma)
h = fspecial('motion',len,theta)
h = fspecial('prewitt')
h = fspecial('sobel')
```

Description

`h = fspecial(type)` creates a two-dimensional filter `h` of the specified type. Some of the filter types have optional additional parameters, shown in the following syntaxes. `fspecial` returns `h` as a correlation kernel, which is the appropriate form to use with `imfilter`.

example

`h = fspecial('average',hsize)` returns an averaging filter `h` of size `hsize`.

`h = fspecial('disk',radius)` returns a circular averaging filter (pillbox) within the square matrix of size `2*radius+1`.

`h = fspecial('gaussian',hsize,sigma)` returns a rotationally symmetric Gaussian lowpass filter of size `hsize` with standard deviation `sigma`. Not recommended. Use `imgaussfilt` or `imgaussfilt3` instead.

`h = fspecial('laplacian',alpha)` returns a 3-by-3 filter approximating the shape of the two-dimensional Laplacian operator, `alpha` controls the shape of the Laplacian.

`h = fspecial('log',hsize,sigma)` returns a rotationally symmetric Laplacian of Gaussian filter of size `hsize` with standard deviation `sigma`.

`h = fspecial('motion',len,theta)` returns a filter to approximate, once convolved with an image, the linear motion of a camera. `len` specifies the length of the motion and `theta` specifies the angle of motion in degrees in a counter-clockwise direction. The filter becomes a vector for horizontal and vertical motions. The default `len` is 9 and the default `theta` is 0, which corresponds to a horizontal motion of nine pixels.

`h = fspecial('prewitt')` returns a 3-by-3 filter that emphasizes horizontal edges by approximating a vertical gradient. To emphasize vertical edges, transpose the filter `h'`.

```
[ 1  1  1
  0  0  0
 -1 -1 -1 ]
```

`h = fspecial('sobel')` returns a 3-by-3 filter that emphasizes horizontal edges using the smoothing effect by approximating a vertical gradient. To emphasize vertical edges, transpose the filter `h'`.

```
[ 1  2  1
  0  0  0
 -1 -2 -1 ]
```

Examples

collapse all

▼ Create Various Filters and Filter an Image

Read image and display it.

Try This Example

 Copy Command

```
I = imread('cameraman.tif');
imshow(I);
```

 Get ▼



Create a motion filter and use it to blur the image. Display the blurred image.

```
H = fspecial('motion',20,45);  
MotionBlur = imfilter(I,H,'replicate');  
imshow(MotionBlur);
```

 Get ▼



Create a disk filter and use it to blur the image. Display the blurred image.

```
H = fspecial('disk',10);  
blurred = imfilter(I,H,'replicate');  
imshow(blurred);
```

 Get ▼



Input Arguments

collapse all

▼ **type — Type of filter**
'average' | 'disk' | 'gaussian' | 'laplacian' | 'log' | 'motion' | 'prewitt' | 'sobel'

Type of filter, specified as one of the following values:

Value	Description
'average'	Averaging filter
'disk'	Circular averaging filter (pillbox)
'gaussian'	Gaussian lowpass filter. Not recommended. Use <code>imgaussfilt</code> or <code>imgaussfilt3</code> instead.
'laplacian'	Approximates the two-dimensional Laplacian operator
'log'	Laplacian of Gaussian filter
'motion'	Approximates the linear motion of a camera
'prewitt'	Prewitt horizontal edge-emphasizing filter
'sobel'	Sobel horizontal edge-emphasizing filter

Data Types: char | string

▼ **hsize — Size of the filter**
positive integer | 2-element vector of positive integers

Size of the filter, specified as a positive integer or 2-element vector of positive integers. Use a vector to specify the number of rows and columns in `h`. If you specify a scalar, then `h` is a square matrix.

When used with the 'average' filter type, the default filter size is `[3 3]`. When used with the Laplacian of Gaussian ('log') filter type, the default filter size is `[5 5]`.

Data Types: double

▼ **radius — Radius of a disk-shaped filter**
5 (default) | positive number

Radius of a disk-shaped filter, specified as a positive number. The filter is a square matrix of size `2*radius+1`.

Data Types: double

▼ **sigma — Standard deviation**
0.5 (default) | positive number

Standard deviation, specified as a positive number.

Data Types: double

▼ **alpha — Shape of the Laplacian**
0.2 (default) | number in the range [0, 1]

Shape of the Laplacian, specified as a number in the range [0, 1]. Specify `alpha` as 0 to obtain a 4-neighborhood Laplacian filter:

```
[ 0  1  0
  1 -4  1
  0  1  0 ]
```

Data Types: double
✓ len — Linear motion of camera 9 (default) numeric scalar
Linear motion of camera, specified as a numeric scalar, measured in pixels. Data Types: double
✓ theta — Angle of camera motion 0 (default) numeric scalar
Angle of camera motion in degrees, specified as a numeric scalar. The angle is measured in a counter-clockwise direction from horizontal. Data Types: double

Output Arguments

collapse all

✓ h — Correlation kernel matrix
Correlation kernel, returned as a matrix. Data Types: double

Algorithms

Averaging filters:

$$\text{ones}(n(1),n(2))/(n(1)*n(2))$$

Gaussian filters:

$$h_g(n_1,n_2) = e^{\frac{-(n_1^2+n_2^2)}{2\sigma^2}}$$
$$h(n_1,n_2) = \frac{h_g(n_1,n_2)}{\sum_{n_1}\sum_{n_2}h_g}$$

Laplacian filters:

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$
$$\nabla^2 = \frac{4}{(\alpha + 1)} \begin{bmatrix} \frac{\alpha}{4} & \frac{1-\alpha}{4} & \frac{\alpha}{4} \\ \frac{1-\alpha}{4} & -1 & \frac{1-\alpha}{4} \\ \frac{\alpha}{4} & \frac{1-\alpha}{4} & \frac{\alpha}{4} \end{bmatrix}$$

Laplacian of Gaussian (LoG) filters:

$$h_g(n_1,n_2) = e^{\frac{-(n_1^2+n_2^2)}{2\sigma^2}}$$
$$h(n_1,n_2) = \frac{(n_1^2 + n_2^2 - 2\sigma^2)h_g(n_1,n_2)}{\sigma^4 \sum_{n_1}\sum_{n_2}h_g}$$

Note that fspecial shifts the equation to ensure that the sum of all elements of the kernel is zero (similar to the Laplace kernel) so that the convolution result of homogeneous regions is always zero.

Motion filters:

1. Construct an ideal line segment with the length and angle specified by the arguments `len` and `theta`, centered at the center coefficient of `h`.
2. For each coefficient location (`i,j`), compute the nearest distance between that location and the ideal line segment.
3. `h = max(1 - nearest_distance,0);`
4. Normalize `h`: `h = h/(sum(h(:)))`

Extended Capabilities

> C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

> GPU Code Generation

Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Version History

Introduced before R2006a

See Also

[conv2](#) | [del2](#) | [edge](#) | [imsharpen](#) | [imfilter](#) | [filter2](#) | [fspecial3](#)

Topics

[Filter Images Using Predefined Filter](#)

[What Is Image Filtering in the Spatial Domain?](#)

